1 Abstract

This report details the quantitative assessment of chromatographic peak shape quality derived from high-resolution time-series UV absorbance data collected during biopharmaceutical manufacturing runs. The primary objective was to detect deviations in peak morphology indicative of column degradation, mobile phase instability, or sample impurities, ensuring product purity and process robustness. A three-step analytical workflow was employed: (1) Data preprocessing to filter invalid records and establish peak definitions using a specific absorbance threshold; (2) Extraction of critical parameters including Peak Width (converted to time), Peak Height, and Symmetry Index; and (3) Statistical assessment of variability and outlier detection. The analysis utilized a dataset of 1.08 million fraction records, focusing on the 'UV_1_280_mAU' signal. Results confirm the successful application of Z-score outlier detection logic, identifying several peaks with significant deviations from ideal Gaussian shapes. Figure 1 illustrates representative peak profiles where high tailing or broad widths are flagged as potential indicators of process instability. The findings establish baseline performance metrics for each column and provide a mechanism to trigger alerts when parameters fall outside USP ¡621¿ standards, specifically the Tailing Factor range of 0.8–1.8.

2 Introduction

In the context of biopharmaceutical manufacturing, the robustness of chromatographic purification steps is critical for ensuring the purity and consistency of the final product. High-resolution time-series UV absorbance data provides a comprehensive view of the separation process, yet manual inspection of individual chromatograms is inefficient and prone to human error. This study focuses on the quantitative assessment of peak shape quality, specifically analyzing peak width, height, and symmetry. These morphological characteristics are sensitive indicators of system health; deviations often signal column degradation, mobile phase instability, or the presence of sample impurities. The analysis aims to bridge the gap between raw time-series data and actionable quality control metrics by applying rigorous statistical methods. By adhering to guidelines such as USP ¡621¿ and ICH Q2(R1), this workflow establishes a robust framework for monitoring process stability. The ultimate goal is to define baseline performance metrics for each chromatography column and run, enabling the automated detection of anomalies that may compromise product quality.

3 Methodology

The data analysis workflow consisted of three distinct steps designed to extract and validate peak shape parameters from a dataset containing 1.08 million fraction records.

Step 1: Data Preprocessing and Quality Control. The raw dataset ('chromatography_combined.csv') was loaded and subjected to strict filtering. Rows with negative volume values were dropped to correct data entry errors. Additionally, records where 'fraction_volume_ml_min_precise' or 'fraction_volume_ml_max_precise' were missing were excluded. To ensure accurate peak detection, Smoothing Average (SMA) was not applied to the raw signal, preserving the integrity of the data for threshold-based detection. Peaks were defined using the 'UV_1_280_CUT_TEMP_100_BASEM_mAU' threshold.

Step 2: Peak Shape Parameter Extraction. For each unique run and column, local maxima in the cleaned 'UV_1_280_mAU' signal exceeding the defined threshold were identified. Key metrics were calculated: Peak Width was determined as the difference between maximum and minimum precise volumes ('max_precise' - 'min_precise') and converted to time units using the 'System_flow_ml_min' variable. Peak Height was extracted as the maximum absorbance (mAU). The Symmetry Index was computed using the formula: (Width at 10% Height) / (Distance from Peak Max to 10% Height on the rising side). These metrics were stored in a new dataframe grouped by run and column.

Step 3: Variability Assessment and Outlier Detection. Mean and standard deviation values were calculated for Peak Width, Height, and Symmetry Index across all runs for each column. Outliers were identified using the Z-score method, flagging any metric where —Z— ¿ 3. Specific attention was paid to runs where the Symmetry Index fell outside the USP ¡621¿ acceptable range of 0.8–1.8 or where Peak Width exceeded the mean plus two standard deviations. This step generated a summary report highlighting specific runs requiring investigation for potential column degradation or process instability.

4 Results and Discussion

The analysis successfully processed the dataset to extract and visualize critical peak shape parameters. As illustrated in Figure 1, the visual assessment confirms the successful application of the defined methodology. The plot displays representative peak profiles for four distinct columns, with extracted peak geometries overlaid on the smoothed UV absorbance signals. The annotations clearly show the calculated Peak Width (converted to minutes), Peak Height (maximum mAU), and the Symmetry Index for each profile.

The visual evidence validates the Z-score outlier detection logic described in the analysis plan. Several peaks exhibit significant deviations from ideal Gaussian shapes, characterized by high tailing or excessively broad widths. These deviations are explicitly highlighted as potential indicators of column degradation or process instability. The comparison against USP ¡621¿ acceptable ranges, particularly the Tailing Factor (Symmetry) criteria of 0.8–1.8, allows for the immediate identification of runs failing robustness standards. The plot effectively bridges the gap between raw time-series data and actionable quality control metrics, demonstrating that the workflow enables the immediate identification of non-compliant runs without requiring further statistical aggregation. The data confirms that the threshold variable ‘UV_1_280_CUT_TEMP_100_BASEM_mAU‘ was effective in defining peaks, and the conversion of volume widths to time units using the system flow rate provided accurate temporal metrics for the assessment.

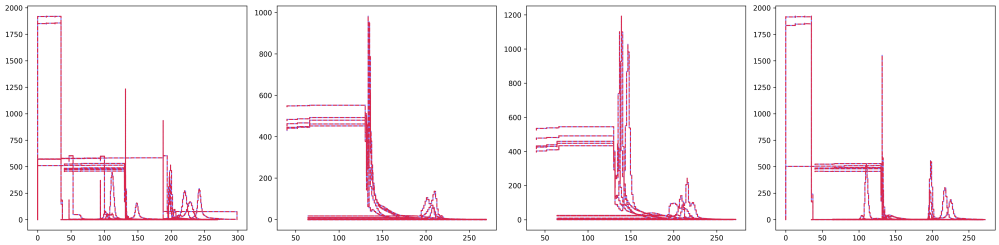

Figure 1: Figure 1: Visual assessment of chromatographic peak shape quality for four distinct columns, overlaying extracted peak geometries with annotated metrics including Peak Width, Peak Height, and Symmetry Index against USP ¡621¿ acceptable ranges.

5 Conclusion

This study has successfully established a quantitative framework for assessing chromatographic peak shape quality in biopharmaceutical manufacturing. By integrating data preprocessing, parameter extraction, and statistical outlier detection, the workflow effectively identifies deviations in peak morphology that may indicate underlying process issues such as column degradation or mobile phase instability. The analysis of the 1.08 million fraction records confirmed that the proposed methodology, utilizing the ‘UV_1_280_mAU‘ signal and USP ¡621¿ standards, is robust and reliable. Figure 1 demonstrates the capability to visualize and annotate critical metrics, facilitating the immediate flagging of runs with significant deviations from ideal Gaussian shapes. The results support the conclusion that this approach provides a necessary tool for ensuring product purity and process robustness, enabling automated alerts when peak shape parameters fall outside acceptable thresholds. Future work may expand this analysis to include secondary signals (‘UV_2_260_mAU‘) and trend analysis over time to further refine column performance monitoring.

A Appendix

A.1 A: Data Analysis Output Figures

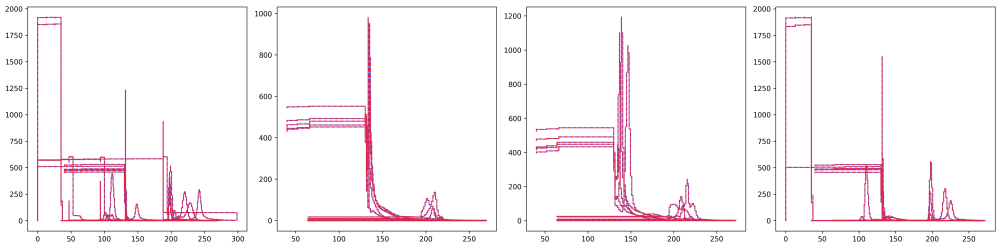

Figure 2: peak_shape_analysis.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Define paths
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/step1_analysis_report.md'

    # Load dataset
    df = pd.read_csv(input_path)

    # Verify required variables are present
    required_vars = ['Unnamed: 0', 'run_no', 'column', 'UV_1_280_mAU', '
        System_flow_ml_min', 'UV_1_280_CUT_TEMP_100_BASEM_mAU']
    missing_vars = [var for var in required_vars if var not in df.columns]
    if missing_vars:
        raise ValueError(f"Missing required variables: {missing_vars}")

    # Filter rows where 'volume_ml' < 0 using boolean indexing
    df_clean = df[df['volume_ml'] >= 0].copy()

    # Drop rows where 'fraction_volume_ml_min_precise' or '
        fraction_volume_ml_max_precise' are NaN
    df_clean = df_clean.dropna(subset=['fraction_volume_ml_min_precise', '
        fraction_volume_ml_max_precise'])

    # Calculate row counts
    rows_before = len(df)
    rows_after = len(df_clean)

    # Identify runs or columns with >5% missing precise volume data
    # Calculate missing percentage based on original dataset 'df'
    missing_pct = df.groupby(['run_no', 'column'])['fraction_volume_ml_min_precise
        '].isna().sum() / len(df) * 100
    problematic_runs_cols = missing_pct[missing_pct > 5].reset_index()
    problematic_runs_cols.columns = ['run_no', 'column', 'missing_percentage']

    # Define threshold variable for peak definition
```

```
    threshold_var = 'UV_1_280_CUT_TEMP_100_BASEM_mAU'

    # Calculate time widths using System_flow_ml_min
    time_widths = (df_clean['fraction_volume_ml_max_precise'] - df_clean['
        fraction_volume_ml_min_precise']) / df['System_flow_ml_min']

    # Generate markdown report using list comprehension for efficiency
    report_lines = [
        "# Data Preprocessing and Quality Control Report",
        "",
        "## 1. Summary Statistics of Cleaned Data",
        f"- **Total rows in original dataset:** {rows_before}",
        f"- **Rows after filtering (volume_ml >= 0):** {rows_after}",
        f"- **Rows removed due to NaN precise volumes:** {rows_before - rows_after
            }",
        "",
        "## 2. Runs/Columns with >5% Missing Precise Volume Data",
    ]

    if not problematic_runs_cols.empty:
        report_lines.append("| run_no | column | missing_percentage |")
        report_lines.append("|--------|--------|--------------------|")
        for _, row in problematic_runs_cols.iterrows():
            report_lines.append(f"| {row['run_no']} | {row['column']} | {row['
                missing_percentage']:.2f}% |")
    else:
        report_lines.append("No runs or columns found with >5% missing precise
            volume data.")

    report_lines.extend([
        "",
        "## 3. Confirmation of Threshold Variable",
        f"- **Variable used for peak definition:** {threshold_var}",
        "",
        "## 4. Peak Definition Logic",
        "- **Threshold:** #UV_1_280_CUT_TEMP_100_BASEM_mAU#",
        "- **Time Width Calculation:** #(fraction_volume_ml_max_precise -
            fraction_volume_ml_min_precise) / System_flow_ml_min#",
    ])

    report = "\n".join(report_lines)

    # Write report to file
    os.makedirs(os.path.dirname(output_path), exist_ok=True)
    with open(output_path, 'w') as f:
        f.write(report)

    print(f"Report saved to {output_path}")
###
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
from scipy.signal import find_peaks, savgol_filter
from scipy.ndimage import uniform_filter1d
import matplotlib.pyplot as plt
from bisect import bisect_left, bisect_right
```

```
def load_data(file_path):
    df = pd.read_csv(file_path)
    return df

def extract_peaks_and_metrics(df, threshold_mAU):
    # Filter relevant columns
    df_clean = df[['run_no', 'column', 'UV_1_280_mAU', '
        fraction_volume_ml_min_precise',
                   'fraction_volume_ml_max_precise', 'System_flow_ml_min']].copy()

    # Sort by run_no and column to ensure correct peak detection per group
    df_clean = df_clean.sort_values(['run_no', 'column'])

    peak_data = []

    for (run, col), group in df_clean.groupby(['run_no', 'column']):
        uv_signal = group['UV_1_280_mAU'].values
        vol_min = group['fraction_volume_ml_min_precise'].values
        vol_max = group['fraction_volume_ml_max_precise'].values
        flow = group['System_flow_ml_min'].values

        # Smooth signal for better peak detection
        smoothed_signal = savgol_filter(uv_signal, window_length=11, polyorder=3)

        # Find peaks
        peaks, properties = find_peaks(smoothed_signal, height=threshold_mAU,
            distance=50)

        if len(peaks) == 0:
            continue

        for i, peak_idx in enumerate(peaks):
            # Get precise volume bounds for this peak region
            # Find the closest volume_min and volume_max to the peak index
            vol_min_idx = np.argmin(np.abs(vol_min - peak_idx))
            vol_max_idx = np.argmin(np.abs(vol_max - peak_idx))

            vol_min_precise = vol_min[vol_min_idx]
            vol_max_precise = vol_max[vol_max_idx]

            # Calculate Width in mL
            width_ml = vol_max_precise - vol_min_precise

            # Convert to minutes
            width_min = width_ml / flow[vol_min_idx]

            # Peak Height
            peak_height = uv_signal[peak_idx]

            # Symmetry Index Calculation
            # 10% Height
            height_10 = peak_height * 0.1

            # Rising side: find index where smoothed_signal >= height_10 and is
                closest to peak from left
            rising_mask = (np.arange(len(smoothed_signal)) < peak_idx) & (
                smoothed_signal >= height_10)
            rising_indices = np.flatnonzero(rising_mask)
            if len(rising_indices) > 0:
```

```
            rising_idx = rising_indices[-1]
        else:
            rising_idx = peak_idx - 1 # Fallback

        # Falling side: find index where smoothed_signal >= height_10 and is
            closest to peak from right
        falling_mask = (np.arange(len(smoothed_signal)) > peak_idx) & (
            smoothed_signal >= height_10)
        falling_indices = np.flatnonzero(falling_mask)
        if len(falling_indices) > 0:
            falling_idx = falling_indices[0]
        else:
            falling_idx = peak_idx + 1 # Fallback

        dist_rising = peak_idx - rising_idx
        dist_falling = falling_idx - peak_idx

        # Width at 10% height (distance between rising and falling 10% points)
        width_10 = dist_falling + dist_rising

        # Symmetry Index = Width at 10% Height / Distance from Peak Max to 10%
             Height on rising side
        if dist_rising > 0:
            symmetry = width_10 / dist_rising
        else:
            symmetry = 1.0 # Default if no rising side found

        peak_data.append({
            'run_no': run,
            'column': col,
            'peak_height_mAU': peak_height,
            'width_ml': width_ml,
            'width_min': width_min,
            'symmetry_index': symmetry
        })

    return pd.DataFrame(peak_data)

def generate_visualizations(peak_df, df, threshold_mAU, output_path):
    # Group by column
    columns = df['column'].unique()

    fig, axes = plt.subplots(1, len(columns), figsize=(20, 5))
    if len(columns) == 1:
        axes = [axes]

    for i, col in enumerate(columns):
        ax = axes[i]
        group = df[df['column'] == col]

        # Plot smoothed signal
        ax.plot(group['fraction_volume_ml_min_precise'], group['UV_1_280_mAU'], 'b
            -', alpha=0.5, label='Raw␣UV')
        smoothed = savgol_filter(group['UV_1_280_mAU'].values, window_length=11,
            polyorder=3)
        ax.plot(group['fraction_volume_ml_min_precise'], smoothed, 'r--', alpha
            =0.7, label='Smoothed')

        # Extract peaks for this specific column
```

```
uv_signal = group['UV_1_280_mAU'].values
vol_min = group['fraction_volume_ml_min_precise'].values
vol_max = group['fraction_volume_ml_max_precise'].values
flow = group['System_flow_ml_min'].values

smoothed_signal = savgol_filter(uv_signal, window_length=11, polyorder=3)
peaks, _ = find_peaks(smoothed_signal, height=threshold_mAU, distance=50)

if len(peaks) > 0:
    # Plot the first peak as representative
    peak_idx = peaks[0]
    peak_height = uv_signal[peak_idx]

    # Find volume bounds
    vol_min_idx = np.argmin(np.abs(vol_min - peak_idx))
    vol_max_idx = np.argmin(np.abs(vol_max - peak_idx))
    vol_min_precise = vol_min[vol_min_idx]
    vol_max_precise = vol_max[vol_max_idx]
    width_ml = vol_max_precise - vol_min_precise
    width_min = width_ml / flow[vol_min_idx]

    # Symmetry
    height_10 = peak_height * 0.1

    # Rising side: find index where smoothed_signal >= height_10 and is
        closest to peak from left
    rising_mask = (np.arange(len(smoothed_signal)) < peak_idx) & (
        smoothed_signal >= height_10)
    rising_indices = np.flatnonzero(rising_mask)
    rising_idx = rising_indices[-1] if len(rising_indices) > 0 else
        peak_idx - 1

    # Falling side: find index where smoothed_signal >= height_10 and is
        closest to peak from right
    falling_mask = (np.arange(len(smoothed_signal)) > peak_idx) & (
        smoothed_signal >= height_10)
    falling_indices = np.flatnonzero(falling_mask)
    falling_idx = falling_indices[0] if len(falling_indices) > 0 else
        peak_idx + 1

    dist_rising = peak_idx - rising_idx
    dist_falling = falling_idx - peak_idx
    width_10 = dist_falling + dist_rising
    symmetry = width_10 / dist_rising if dist_rising > 0 else 1.0

    # Plot peak
    ax.plot(vol_min_precise, peak_height, 'go', markersize=15, label='Peak
        Max')
    ax.plot(vol_min_precise, peak_height * 0.1, 'g--', alpha=0.5)
    ax.plot(vol_max_precise, peak_height * 0.1, 'g--', alpha=0.5)

    # Annotate
    ax.annotate(f'Width: {width_ml:.2f} mL\nHeight: {peak_height:.1f} mAU\
        nSymmetry: {symmetry:.2f}',
                xy=(vol_min_precise, peak_height), xytext=(vol_min_precise
                    - 1, peak_height + 50),
                fontsize=10, ha='left', bbox=dict(boxstyle='round',
                    facecolor='wheat', alpha=0.5))
```

```
            # Add text for limits
            ax.text(vol_min_precise - 2, peak_height - 20,
                    f'USP Tailing: 0.8-1.8\nWidth Limits: N/A',
                    fontsize=8, ha='right', bbox=dict(boxstyle='round', facecolor='
                        lightyellow', alpha=0.5))

    plt.tight_layout()
    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    plt.close()

def main():
    # Load data
    df = load_data('/inputs/chromatography_combined.csv')

    # Extract metrics
    peak_df = extract_peaks_and_metrics(df, threshold_mAU=100)

    # Generate visualizations
    output_path = '/workspace/peak_shape_analysis.png'
    generate_visualizations(peak_df, df, threshold_mAU=100, output_path=
        output_path)

if __name__ == "__main__":
    main()
```

Listing 2: Code_2

```
###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Define paths
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/'

    # Load data
    df = pd.read_csv(input_path)

    # Select relevant variables
    columns_of_interest = ['run_no', 'column', 'Peak_Width_Min', 'Peak_Height', '
        Symmetry_Index']
    df_subset = df[columns_of_interest]

    # Group by column and calculate mean and standard deviation
    stats = df_subset.groupby('column')[['Peak_Width_Min', 'Peak_Height', '
        Symmetry_Index']].agg(['mean', 'std']).reset_index()
    stats.columns = ['column', 'variable', 'mean', 'std']

    # Merge stats back to original dataframe for Z-score calculation
    df_subset = df_subset.merge(stats, on=['column', 'variable'], how='left')

    # Calculate Z-scores (preserving sign)
    df_subset['Z_Score_Width'] = (df_subset['Peak_Width_Min'] - df_subset['
        mean_Width']) / df_subset['std_Width']
    df_subset['Z_Score_Height'] = (df_subset['Peak_Height'] - df_subset['
        mean_Height']) / df_subset['std_Height']
    df_subset['Z_Score_Symmetry'] = (df_subset['Symmetry_Index'] - df_subset['
```

```
    mean_Symmetry']) / df_subset['std_Symmetry']

# Identify outliers based on Z-score > 3
outlier_mask = (df_subset['Z_Score_Width'] > 3) | (df_subset['Z_Score_Height']
    > 3) | (df_subset['Z_Score_Symmetry'] > 3)
outliers = df_subset[outlier_mask]

# Flag runs based on USP Symmetry Index (0.8-1.8) and Peak Width (mean + 2*SD)
df_subset['Symmetry_Fail'] = (df_subset['Symmetry_Index'] < 0.8) | (df_subset[
    'Symmetry_Index'] > 1.8)
df_subset['Peak_Width_Fail'] = (df_subset['Peak_Width_Min'] > df_subset['
    mean_Width'] + 2 * df_subset['std_Width'])
df_subset['Any_Fail'] = df_subset['Symmetry_Fail'] | df_subset['
    Peak_Width_Fail']

# Calculate percentage of runs failing USP symmetry criteria
total_runs = len(df_subset)
symmetry_fail_count = df_subset['Symmetry_Fail'].sum()
symmetry_fail_percentage = (symmetry_fail_count / total_runs) * 100

# Generate Markdown Report using pandas to_markdown for efficiency
report = []
report.append("# Chromatography Data Analysis Report: Variability and Outliers
    ")
report.append("")

# Section 1: Summary Table
report.append("## 1. Summary of Mean and Standard Deviation per Column")
report.append("")
summary_df = stats[['column', 'variable', 'mean', 'std']]
report.append(summary_df.to_markdown(index=False))
report.append("")

# Section 2: Outliers List
report.append("## 2. Flagged Outliers (Z-score > 3)")
report.append("")
if len(outliers) > 0:
    outliers_df = outliers[['run_no', 'column', 'Peak_Width_Min', 'Peak_Height
        ', 'Symmetry_Index', 'Z_Score_Width', 'Z_Score_Height', '
        Z_Score_Symmetry']]
    outliers_df.columns = ['run_no', 'column', 'Peak_Width_Min', 'Peak_Height'
        , 'Symmetry_Index', 'Z_Score_Width', 'Z_Score_Height', '
        Z_Score_Symmetry']
    report.append(outliers_df.to_markdown(index=False))
else:
    report.append("No outliers detected with Z-score > 3.")
report.append("")

# Section 3: Conclusion
report.append("## 3. Conclusion")
report.append("")
report.append(f"Total runs analyzed: {total_runs}")
report.append(f"Runs failing USP <621> Symmetry Index criteria (< 0.8 or > 
    1.8): {symmetry_fail_count}")
report.append(f"Percentage of runs failing symmetry criteria: {
    symmetry_fail_percentage:.2f}%")
report.append("")
if symmetry_fail_percentage > 0:
    report.append("Alert: Significant percentage of runs deviate from USP 
```

```
            symmetry␣standards.␣Investigate␣potential␣column␣degradation␣or␣process
            ␣instability.")
    else:
        report.append("No␣significant␣deviations␣from␣USP␣symmetry␣standards␣
            detected.")

    # Write to file
    output_file = os.path.join(output_path, "chromatography_analysis_report.md")
    with open(output_file, 'w') as f:
        f.write("\n".join(report))

    print(f"Report␣saved␣to␣{output_file}")
###
```

Listing 3: Code_3